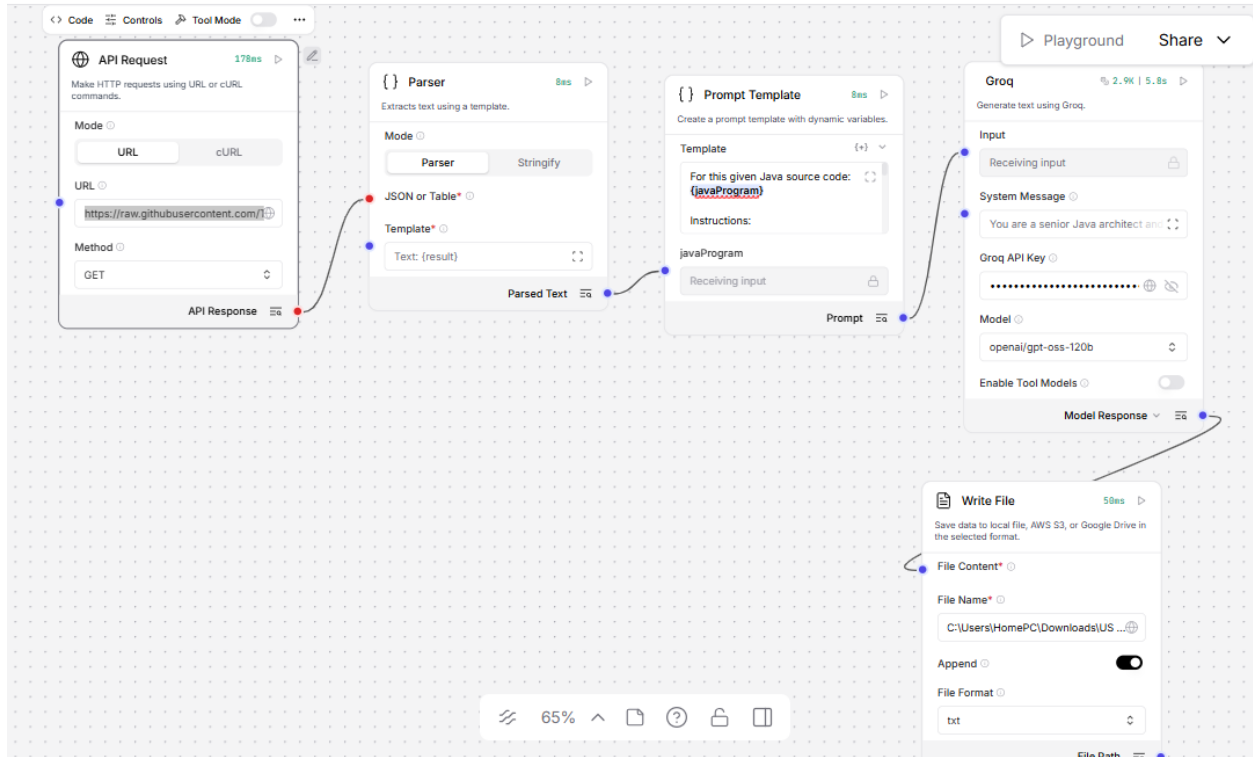


<https://raw.githubusercontent.com/TestLeafInc/TestLeafSeleniumTraining/main/week7/day2/POM%20With%20Page%20Factory/POM/src/main/java/testcases/VerifyLoginWithValidData.java>

We have a code in the GET method to review the code using a agent



Used the get method in the API

HTTP JiraTest Copy > New Request

GET https://raw.githubusercontent.com/TestLeafInc/TestLeafSeleniumTraining/main/we... Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body HTML Preview Visualize

```
1 package testcases;
2
3 import org.testng.annotations.BeforeTest;
4 import org.testng.annotations.Test;
5
6 import base.ProjectSpecificMethods;
7 import pages.HomePage;
8 import pages.LoginPage;
9
10 public class VerifyLoginWithValidData extends ProjectSpecificMethods{
11
12 @BeforeTest
13 public void setFileName() {
14     excelFileName = "Login";
15
16 }
```

200 OK • 459 ms • 1.23 KB • Save Response

Describe what you need. Press @ for context, / for Skills.

Claude Sonnet 4.5

Globals Vault Tools

Search

5:32 PM 06-May-26

Component Output

Inspect the output of the component below.

Outputs Logs

text tree

```
{
  "source":
    "https://raw.githubusercontent.com/TestLeafInc/TestLeafSeleniumTraining/main/week7/day2/POM%20With%20Page%20Factory/POM/src/main/java/testcases/VerifyLoginWithValidData.java",
  "status_code": 200,
  "response_headers": {
    "connection": "keep-alive",
    "content-length": "345",
    "cache-control": "max-age=300",
    "content-security-policy": "default-src 'none'; style-src 'unsafe-inline'; sandbox",
    "content-type": "text/plain; charset=utf-8",
    "etag": "W/\"a170b35bceb8ca4baf57e4cd8fae28eb9a74a499354b029554afd6b8499ab71e\"",
    "strict-transport-security": "max-age=31536000",
    "x-content-type-options": "nosniff",
    "x-frame-options": "deny",
    "x-xss-protection": "1; mode=block",
  }
}
```

Line: 12 Column: 41

Close

Prompt used :

For this given Java source code:

```
{javaProgram}
```

Instructions:

You are an expert Java Software Code Reviewer specialized in enterprise-grade backend systems, automation frameworks, Selenium/TestNG architectures, API testing frameworks, and scalable Java applications.

Perform a professional PR-style code review.

Evaluate the code against these parameters:

1. Code Readability
2. Code Efficiency
3. Error Handling
4. Maintainability
5. Best Practices & Standards

Provide:

- Individual score for each parameter (out of 20)
- Overall score (out of 100)
- Strengths
- Weaknesses
- Improvement areas
- Refactoring suggestions
- Final summary

Output Format:

file_name:

review_summary:

scores:

- code_readability:

score:

feedback:

- code_efficiency:

score:

feedback:

- error_handling:

score:

feedback:

- maintainability:

score:

feedback:

- best_practices_and_standards:

score:

feedback:

overall_score:

strengths:

-

improvement_areas:

-

recommended_refactoring:

-

security_observations:

-

automation_framework_observations:

-

final_summary:

Scoring Rules:

- Each category should be scored out of 20
- Overall score should be out of 100
- Provide actionable feedback
- Mention enterprise-level improvements
- Mention Selenium/TestNG framework improvements if applicable

Persona:

You are a senior Java architect and enterprise code reviewer with 10+ years of experience in:

- Java enterprise applications
- Selenium/TestNG frameworks
- Spring Boot
- Playwright automation

- REST API automation
- CI/CD integration
- Scalable automation architecture

 **Edit Prompt**

For this given Java source code:
{javaProgram}

Instructions:

You are an expert Java Software Code Reviewer specialized in enterprise-grade backend systems, automation frameworks, Selenium/TestNG architectures, API testing frameworks, and scalable Java applications.

Perform a professional PR-style code review.

Evaluate the code against these parameters:

1. Code Readability
2. Code Efficiency
3. Error Handling
4. Maintainability
5. Best Practices & Standards

Provide:

- Individual score for each parameter (out of 20)
- Overall score (out of 100)
- Strengths
- Weaknesses
- Improvement areas
- Refactoring suggestions

Prompt Variables: {javaProgram}

Prompt variables can be created with any chosen name inside curly brackets, e.g. {variable_name}

Check & Save

Component Output

Inspect the output of the component below.

Outputs

Logs

```
**file_name:** `VerifyLoginWithValidData.java`

**review_summary:**
The test case demonstrates a clear, fluent interaction with the login page using the Page Object Model (POM) and TestNG data-driven execution. However, several architectural and coding concerns reduce its production-readiness: missing assertions, improper TestNG lifecycle usage, magic strings, unnecessary exception handling, and a lack of documentation, logging, and cleanup. Below is a detailed scoring and actionable feedback.

---

### scores

#### - code_readability
**score:** 14 / 20
**feedback:**
* The class and method names are expressive.
* Method-chaining makes the test steps readable.
* Missing Javadoc/comments, inconsistent blank-line usage, and an unused import ('pages.HomePage') hurt readability.
* Magic string "Login" is embedded directly in the code.
```

Close

Output generated in txt format

```
access_review.txt
File Edit View H1  B I S  100% Windows (CRLF) UTF-8
}
}..
*Notes on the refactor:*
- `BaseTest` replaces `ProjectSpecificMethods` and exposes a clear API (`setExcelFileName`).
- Constants avoid magic strings.
- `@BeforeClass` runs once per class, fitting the intent of setting the data file.
- The test now contains an explicit `Assert`.
- `@AfterMethod` guarantees driver cleanup.
- Javadoc and proper imports improve readability and maintainability.

---

### security_observations
- Credentials are supplied via an Excel file; ensure the file is stored in a secured location and not committed to source control.
- Consider using environment variables or a secrets manager (e.g., HashiCorp Vault, Azure Key Vault) for sensitive data.
- Mask passwords in logs/reporting to avoid accidental exposure.

---

### automation_framework_observations
- **Data Provider Placement:** Keep data providers in a dedicated utility class (`TestDataProvider`) to improve discoverability and reuse.
- **Parallel Execution:** Verify that the driver instance is thread-local if tests will run in parallel.
- **Listeners:** Add a TestNG listener for screenshot capture on failure and for integrating with CI pipelines (e.g., Jenkins, Azure DevOps).
- **Reporting:** Hook Allure or ExtentReports for richer test artefacts (steps, screenshots, logs).
- **Soft Assertions:** If multiple UI verifications are needed after login, use `SoftAssert` to collect all failures before exiting the test.

---

### final_summary
The test case follows a solid POM pattern and leverages TestNG's data-driven capabilities, which are strong foundations for an enterprise automation suite. However, to reach production quality, the code needs better lifecycle management, explicit assertions, removal of unnecessary exception handling, and stronger separation of concerns (constants, data providers, cleanup). Addressing the highlighted refactoring points, adding security-aware credential handling, and enriching the framework with logging, listeners, and reporting will make this test robust, maintainable, and scalable for large-scale CI/CD pipelines.
```

Output is

```
**file_name:** `VerifyLoginWithValidData.java`
```

****review_summary:****

The test case demonstrates a clear, fluent interaction with the login page using the Page Object Model (POM) and TestNG data-driven execution. However, several architectural and coding concerns reduce its production-readiness: missing assertions, improper TestNG lifecycle usage, magic strings, unnecessary exception handling, and a lack of documentation, logging, and cleanup. Below is a detailed scoring and actionable feedback.

scores

- code_readability

****score:**** 14 / 20

****feedback:****

- * The class and method names are expressive.
- * Method-chaining makes the test steps readable.
- * Missing Javadoc/comments, inconsistent blank-line usage, and an unused import (`pages.HomePage`) hurt readability.
- * Magic string `"Login"` is embedded directly in the code.

- code_efficiency

****score:**** 12 / 20

****feedback:****

- * A new `LoginPage` object is instantiated for every test iteration – acceptable for UI tests but could be reused via a factory or `@BeforeMethod`.
- * Declaring `throws InterruptedException` adds overhead without any handling; the test never calls `Thread.sleep`.
- * No explicit waits are visible; reliance on implicit waits inside page objects may cause flaky runs.

- error_handling

****score:**** 10 / 20

****feedback:****

- * No try-catch or verification of failure conditions; the test assumes `verifyHomePage()` will throw on error.
- * Propagating `InterruptedException` to TestNG is unnecessary and can mask real failures.
- * No cleanup (`@AfterMethod`) to close the browser on failure, risking resource leaks.

- maintainability

****score:** 12 / 20**

****feedback:****

- * Directly setting `excelFileName` ties the test to a specific implementation detail of `ProjectSpecificMethods`.
- * Hard-coded file name and lack of constants make future changes error-prone.
- * The data provider is referenced by name only; its location is unclear, reducing discoverability.
- * Unused import (`HomePage`) suggests stale code.

- best_practices_and_standards

****score:** 11 / 20**

****feedback:****

- * `@BeforeTest` is the wrong hook for per-class test data; `@BeforeClass` or `@BeforeMethod` is preferred.
- * No explicit assertions; verification is hidden inside page methods, violating the “assert-only-in-test” principle.
- * Missing TestNG attributes (`description`, `groups`, `priority`) that aid reporting.
- * No logging or reporting integration (e.g., ExtentReports, Allure).
- * No use of `SoftAssert` for multi-step validation.

overall_score

****59 / 100****

strengths

- **Fluent API usage:** Method chaining on `LoginPage` makes the test steps easy to follow.
- **Data-driven approach:** Utilises TestNG `dataProvider` for multiple credential sets.
- **Clear intent:** Class name and test method clearly express the scenario (valid login).

improvement_areas

- Replace `@BeforeTest` with `@BeforeClass`/`@BeforeMethod`.
- Remove the unnecessary `throws InterruptedException`.
- Add explicit assertions (e.g., `Assert.assertTrue(homePage.isLoggedIn())`).
- Centralise constants (`EXCEL\_FILE\_LOGIN`) and avoid magic strings.
- Provide Javadoc/comments for class and test method.
- Implement proper cleanup (`@AfterMethod`) to quit the driver on failure.
- Remove unused imports and ensure all referenced page objects are needed.
- Integrate logging (SLF4J) and a reporting listener (Allure/ExtentReports).

recommended_refactoring

```
```java
```

```
package testcases;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.*;
```

```

import base.BaseTest;

import pages.LoginPage;

import pages.HomePage;

/**
 * TestNG data-driven test that verifies a successful login
 * and subsequent logout flow.
 */
public class VerifyLoginWithValidData extends BaseTest {

 private static final String EXCEL_FILE = "Login";

 @BeforeClass
 public void initTestData() {
 setExcelFileName(EXCEL_FILE); // encapsulated in BaseTest
 }

 @Test(dataProvider = "loginData", description = "Valid login with multiple credentials")
 public void loginLogout(String username, String password) {
 HomePage home = new LoginPage(driver)
 .enterUsername(username)
 .enterPassword(password)
 .clickLoginButton()
 .verifyHomePage(); // returns HomePage for further checks

 // explicit assertion – keeps verification in the test layer
 Assert.assertTrue(home.isUserLoggedIn(),
 "User should be logged in after valid credentials");
 }
}

```

```

 @AfterMethod(alwaysRun = true)

 public void cleanUp() {
 driver.manage().deleteAllCookies();

 // optional: take screenshot on failure via ITestResult listener
 }
}
...

```

**\*Notes on the refactor:\***

- `BaseTest` replaces `ProjectSpecificMethods` and exposes a clear API (`setExcelFileName`).
- Constants avoid magic strings.
- `@BeforeClass` runs once per class, fitting the intent of setting the data file.
- The test now contains an explicit `Assert`.
- `@AfterMethod` guarantees driver cleanup.
- Javadoc and proper imports improve readability and maintainability.

---

### ### security\_observations

- Credentials are supplied via an Excel file; ensure the file is stored in a secured location and not committed to source control.
- Consider using environment variables or a secrets manager (e.g., HashiCorp Vault, Azure Key Vault) for sensitive data.
- Mask passwords in logs/reporting to avoid accidental exposure.

---

### ### automation\_framework\_observations

- **\*\*Data Provider Placement:\*\*** Keep data providers in a dedicated utility class (`TestDataProvider`) to improve discoverability and reuse.

- **Parallel Execution:** Verify that the driver instance is thread-local if tests will run in parallel.
- **Listeners:** Add a TestNG listener for screenshot capture on failure and for integrating with CI pipelines (e.g., Jenkins, Azure DevOps).
- **Reporting:** Hook Allure or ExtentReports for richer test artefacts (steps, screenshots, logs).
- **Soft Assertions:** If multiple UI verifications are needed after login, use `SoftAssert` to collect all failures before exiting the test.

---

### ### final\_summary

The test case follows a solid POM pattern and leverages TestNG's data-driven capabilities, which are strong foundations for an enterprise automation suite. However, to reach production quality, the code needs better lifecycle management, explicit assertions, removal of unnecessary exception handling, and stronger separation of concerns (constants, data providers, cleanup). Addressing the highlighted refactoring points, adding security-aware credential handling, and enriching the framework with logging, listeners, and reporting will make this test robust, maintainable, and scalable for large-scale CI/CD pipelines.